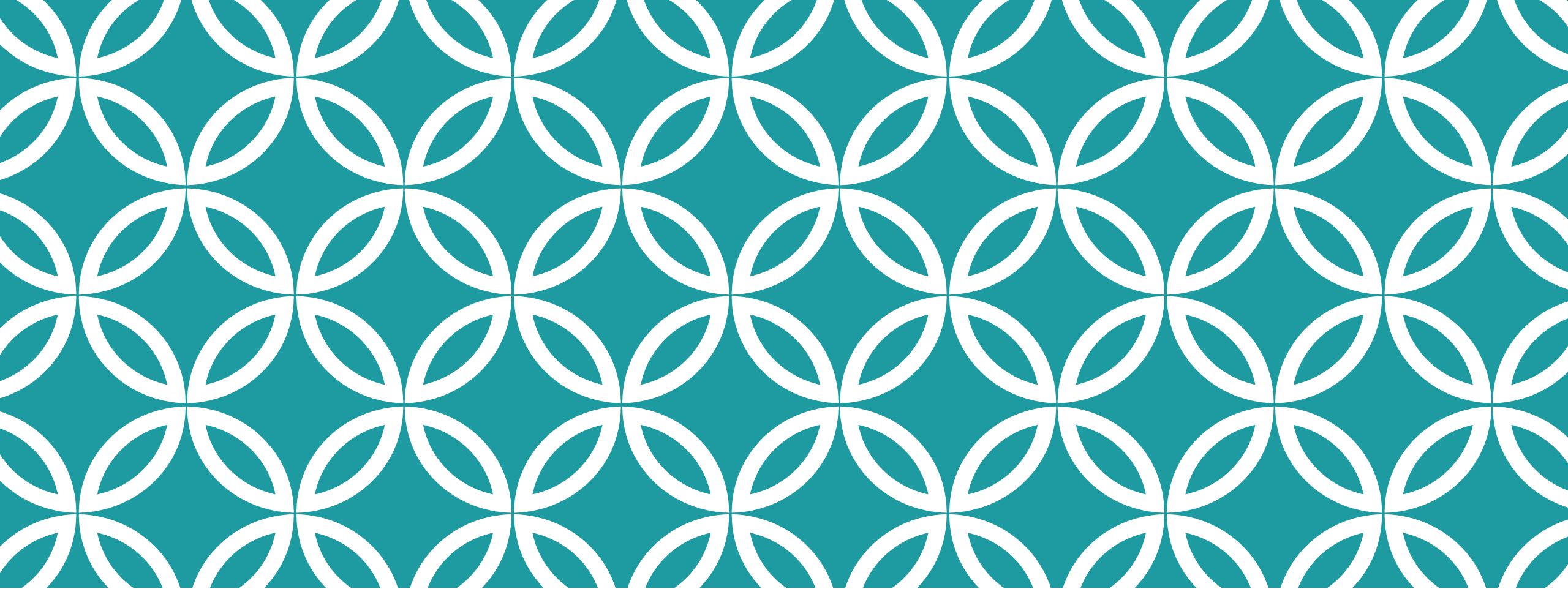




GCC/GDB



GCC 소개

- gcc의 특징

gnu에서 만든 C 컴파일러

/usr/bin/gcc is not compiler

많은 CPU 아키텍처 지원

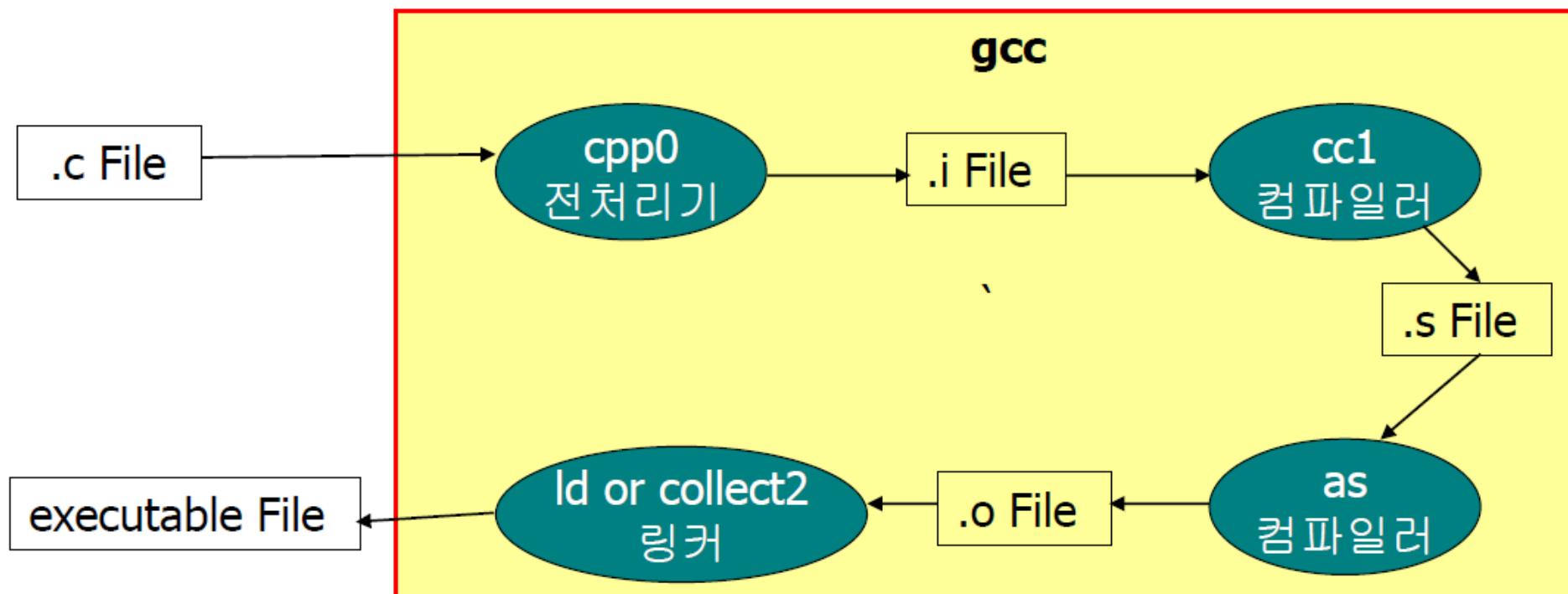
ARM, I386, PPC, SPARC...

다른 언어(C++, fortran...)로 작성된 소스도 컴파일 할 수 있음.

GCC 소개

- gcc의 특징

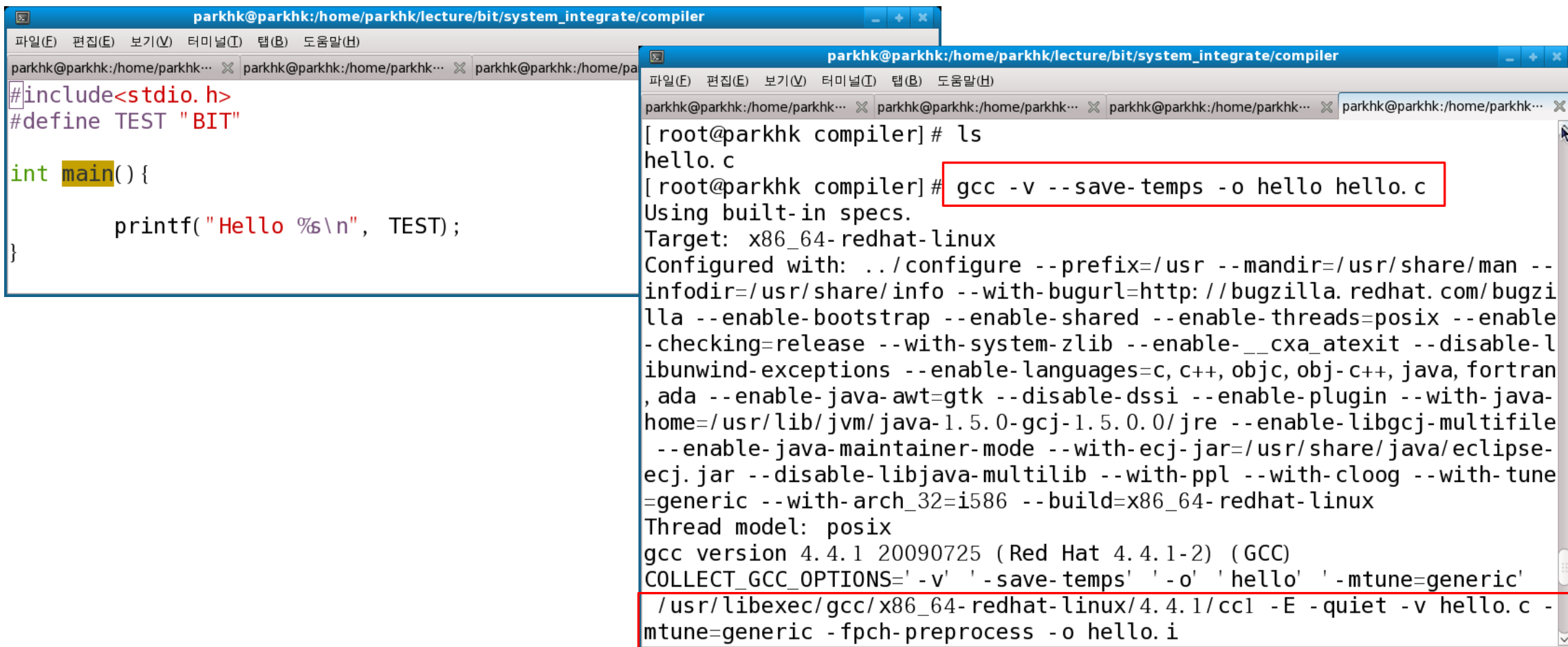
전처리기, C 컴파일러, 어셈블러, 링커를 호출하는 역할 담당



*gcc -v --save-temps 옵션으로 중간 단계 확인

GCC 소개

■ gcc 컴파일 과정 예



The image shows two terminal windows from a presentation. The left window displays a C source file named `hello.c` with the following content:

```
#include<stdio.h>
#define TEST "BIT"

int main() {
    printf("Hello %s\n", TEST);
}
```

The right window shows the execution of the `gcc` compiler. It starts with a `ls` command listing `hello.c`, followed by the command `gcc -v --save-temps -o hello hello.c` (highlighted with a red box). The output shows the target architecture as `x86_64-redhat-linux` and the GCC version as `4.4.1`. The final line of the output, which is also highlighted with a red box, shows the actual compilation command:

```
/usr/libexec/gcc/x86_64-redhat-linux/4.4.1/cc1 -E -quiet -v hello.c -mtune=generic -fpch-preprocess -o hello.i
```

GCC 소개

■ gcc 컴파일 과정 예



```
parkhk@parkhk:/home/parkhk/lecture/bit/system_integrate/compiler
/usr/libexec/gcc/x86_64-redhat-linux/4.4.1/cc1 -fpreprocessed hello.i
-quiet -dumpbase hello.c -mtune=generic -auxbase hello -version -o he
llo.s
GNU C (GCC) version 4.4.1 20090725 (Red Hat 4.4.1-2) (x86_64-redhat-li
nux)
    compiled by GNU C version 4.4.1 20090725 (Red Hat 4.4.1-2)  GM
P version 4.2.4, MPFR version 2.4.1.
GCC heuristics: --param ggc-min-expand=100 --param
1072
Compiler executable checksum: f2d7eb8fb5f7081c13487
COLLECT_GCC_OPTIONS='-v' '-save-temps' '-o' 'hello'
as -V -Qy -o hello.o hello.s

parkhk@parkhk:/home/parkhk/lecture/bit/system_integrate/compiler
/usr/libexec/gcc/x86_64-redhat-linux/4.4.1/collect2 --eh-frame-hdr --
build-id -m elf_x86_64 --hash-style=gnu -dynamic-linker /lib64/ld-linu
x-x86-64.so.2 -o hello /usr/lib/gcc/x86_64-redhat-linux/4.4.1/../../../../
lib64/crt1.o /usr/lib/gcc/x86_64-redhat-linux/4.4.1/../../../../li
b64/crti.o /usr/lib/gcc/x86_64-redhat-linux/4.4.1/crtbegin.o -L/usr/li
b/gcc/x86_64-redhat-linux/4.4.1 -L/usr/lib/gcc/x86_64-redhat-linux/4.4
.1 -L/usr/lib/gcc/x86_64-redhat-linux/4.4.1/../../../../lib64 -L/lib/.
./lib64 -L/usr/lib/./lib64 -L/usr/lib/gcc/x86_64-redhat-linux/4.4.1/.
./../../../../hello.o -lgcc --as-needed -lgcc_s --no-as-needed -lc -lgcc --a
s-needed -lgcc_s --no-as-needed /usr/lib/gcc/x86_64-redhat-linux/4.4.1
/crtend.o /usr/lib/gcc/x86_64-redhat-linux/4.4.1/../../../../lib64/crt
n.o
[root@parkhk compiler] #
```

GCC 소개

■ gcc 컴파일 과정 예

```
parkhk@parkhk:/home/parkhk/lecture/bit/system_integrate/compiler
[root@parkhk compiler]# ls
hello hello.c hello.i hello.o hello.s
[root@parkhk compiler]#
```

```
parkhk@parkhk:/home/parkhk/lecture/bit/system_integrate/compiler
# 2 "hello.c" 2

int main() {
    printf("Hello %s\n", "BIT");
}
```

839, 1 바닥

hello.i

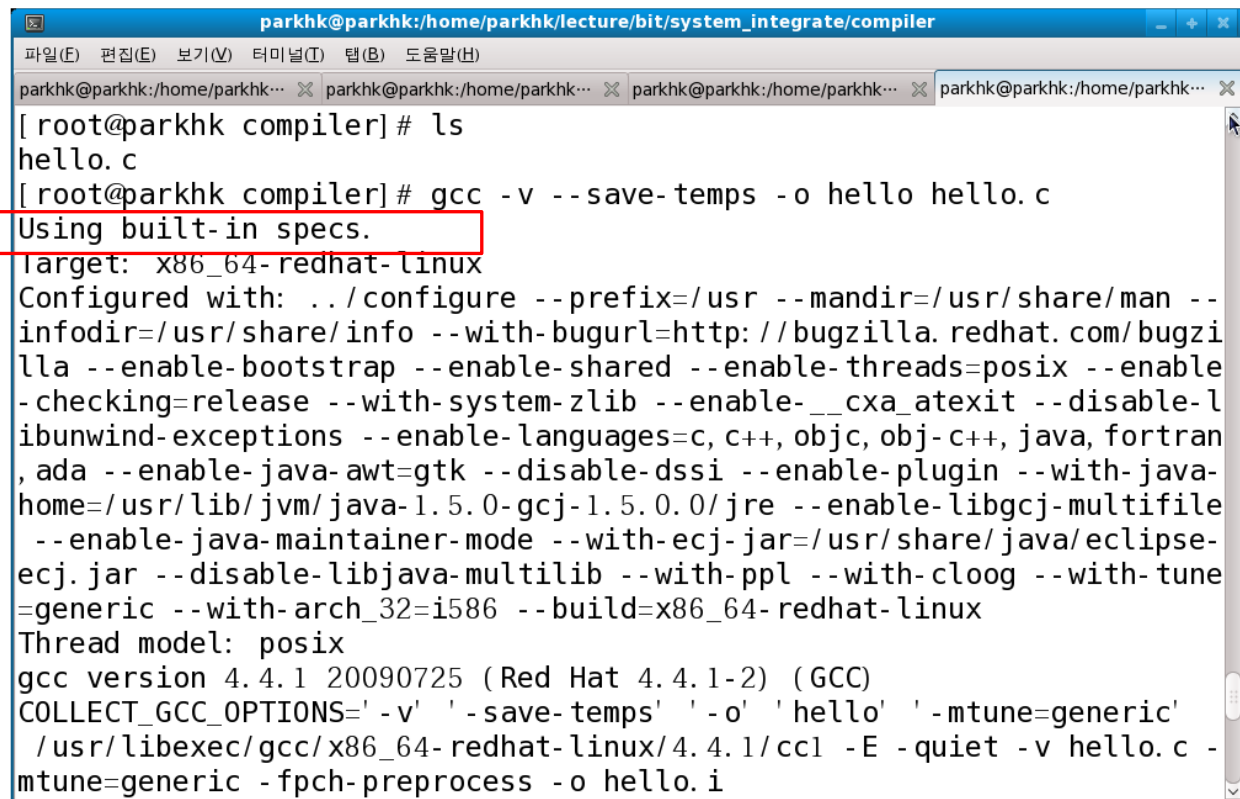
```
parkhk@parkhk:/home/parkhk/lecture/bit/system_in
.LFB0:
    .cfi_startproc
    pushq    %rbp
    .cfi_def_cfa_offset 16
    movq     %rsp, %rbp
    .cfi_offset 6, -16
    .cfi_def_cfa_register 6
    movl     $.LC0, %eax
    movl     $.LC1, %esi
    movq     %rax, %rdi
    movl     $0, %eax
    call     printf
    leave
    ret
    .cfi_endproc
                                11, 6      71%
```

hello.s

GCC 전처리 과정

- spec이란?
 - gcc 컴파일시 생성
 - gcc가 전처리거나 C컴파일러, 어셈블러, 링크를 호출할 때 각각에 적용할 기본적인 옵션들에 대해서 정의해 놓은 파일

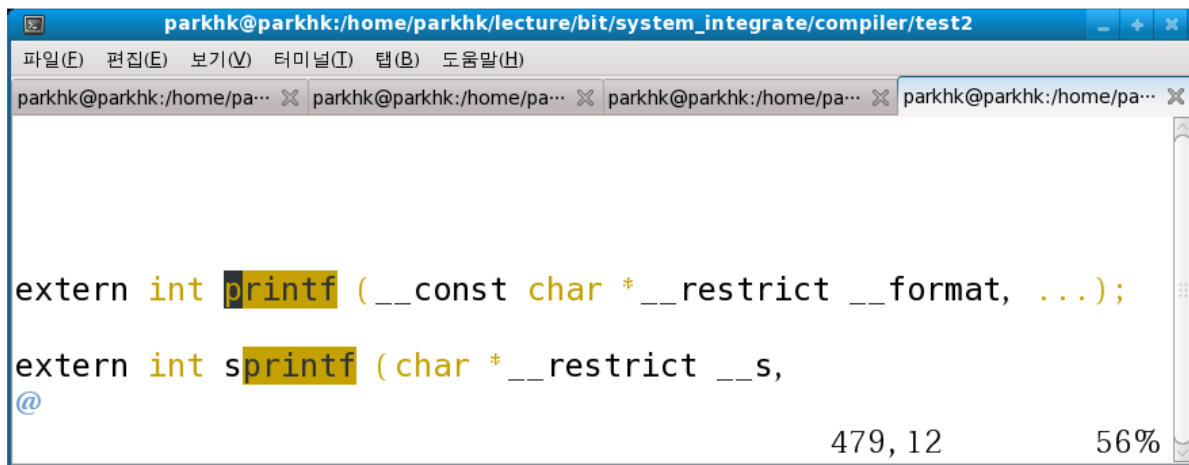
\$ gcc -dumpspecs



```
parkhk@parkhk:/home/parkhk/lecture/bit/system_integrate/compiler
[ root@parkhk compiler]# ls
hello.c
[ root@parkhk compiler]# gcc -v --save-temps -o hello hello.c
Using built-in specs.
target: x86_64-redhat-linux
Configured with: ../configure --prefix=/usr --mandir=/usr/share/man --
infodir=/usr/share/info --with-bugurl=http://bugzilla.redhat.com/bugzi
lla --enable-bootstrap --enable-shared --enable-threads=posix --enable
-checking=release --with-system-zlib --enable-__cxa_atexit --disable-l
ibunwind-exceptions --enable-languages=c,c++,objc,obj-c++,java,fortran
,ada --enable-java-awt=gtk --disable-dssi --enable-plugin --with-java-
home=/usr/lib/jvm/java-1.5.0-gcj-1.5.0.0/jre --enable-libgcj-multifile
--enable-java-maintainer-mode --with-ecj-jar=/usr/share/java/eclipse-
ecj.jar --disable-libjava-multilib --with-ppl --with-cloog --with-tune
=generic --with-arch_32=i586 --build=x86_64-redhat-linux
Thread model: posix
gcc version 4.4.1 20090725 (Red Hat 4.4.1-2) (GCC)
COLLECT_GCC_OPTIONS='-v' '-save-temps' '-o' 'hello' '-mtune=generic'
/usr/libexec/gcc/x86_64-redhat-linux/4.4.1/cc1 -E -quiet -v hello.c -
mtune=generic -fpch-preprocess -o hello.i
```

GCC 전처리 과정

- cpp0 – 전처리 명령어
 - 전처리 수행 과정
 - 1.헤더 파일의 삽입
 - 2.매크로 치환 및 적용



```
extern int printf (__const char *__restrict __format, ...);
extern int sprintf (char *__restrict __s,
```

hello.i

〈include 과정〉

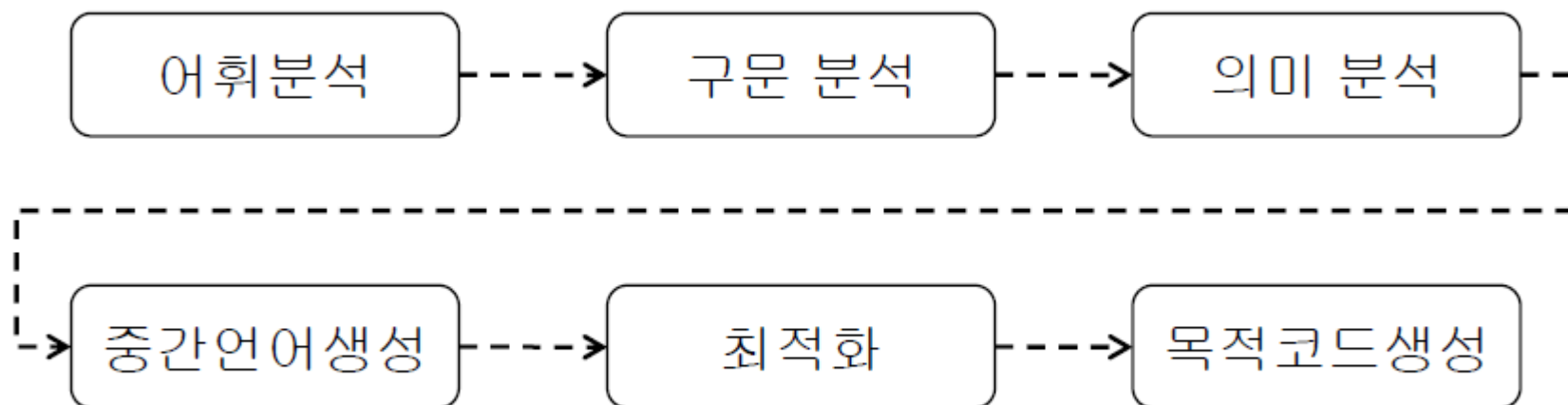
- 1.소스 파일 읽음
- 2.include를 만나면 include 디렉토리에서 헤더파일을 찾음
- 3.찾는 순서 : /usr/local/include => /usr/lib/gcc/*-linux/ gcc버전/include => /usr/include

〈매크로 치환 과정〉

- 1.define된 부분은 cc1의 심볼 테이블에 저장
- 2.심볼 테이블에 들어 있는 문자열과 같은 문자열을 만나게 되면 치환

GCC 컴파일 과정

- cc1에 의한 컴파일 과정



GCC 컴파일 과정

■ 어휘 분석

- 전처리 결과 파일 읽음
- 문법적으로 의미 있는 가장 최소 단위(토큰)로 나눔

종류	예
지정어	If, while, for, do, switch, case 등
연산 기호	+, -, *, /, %, < 등
구분자	(, [, ,, ; 등
명칭	main, printf 등
상수	123, 123.1, 0x123, "bit\ n" 등

- 나누어진 토큰을 파서(구문분석 담당)에게 넘김

GCC 컴파일 과정

■ 구문 분석

- 파서에 의해 구문 분석이 수행됨
- 문법적 오류가 있는지 검사한 후에 중간 언어 생성을 위한 파서 트리 생성
- 의미 분석
- 문법상 오류는 없지만 의미상 오류가 있는 가를 검사
- 예) 선언되지 않은 변수의 사용, 함수의 인자 개수나 인자형의 불 일치등..

■ 중간 언어 생성

- 최적화를 위한 중간 언어 생성
gcc는 RTL(Register Transfer Language)라는 중간 언어를 생성

■ 최적화

GCC 컴파일 과정

■ 최적화

• 최적화의 의미

- 기능은 같지만 크기가 작고, 빠르며, 메모리를 작게 사용하는 코드로 변경하는 것을 의미
- gcc는 컴파일 시간의 대부분을 최적화과정을 위해 사용(소스의 대부분도 최적화를 위한 코드임)

• -O0 ~ -O3 옵션을 통해 최적화 수위 조절

• 최적화 과정

1) 중간 코드 최적화

- 지역 최적화 : 상수 계산 최적화등
- 전역 최적화 : 공통 부분식 제거 및 사용되지 않는 코드 제거
- 루프 최적화 : 루프가 필요 없는 부분은 루프를 제거 또는 합침

2) 목적 코드 최적화

- 레지스터 사용 최적화
- 메모리 접근에 대한 최적화

GCC 컴파일 과정

■ 목적 코드 생성

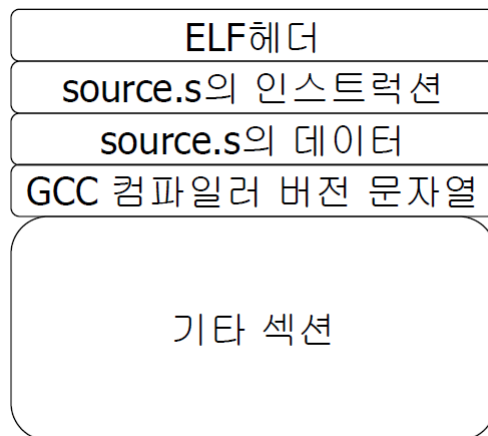
목적코드는 어셈블리 언어로써 이후 `as`에 의해 인스트럭션으로 변환됨

■ 기계어 코드 생성

- `as`에 의해 기계어로 어셈블됨

```
as -V -Qy -o source.o source.s
```

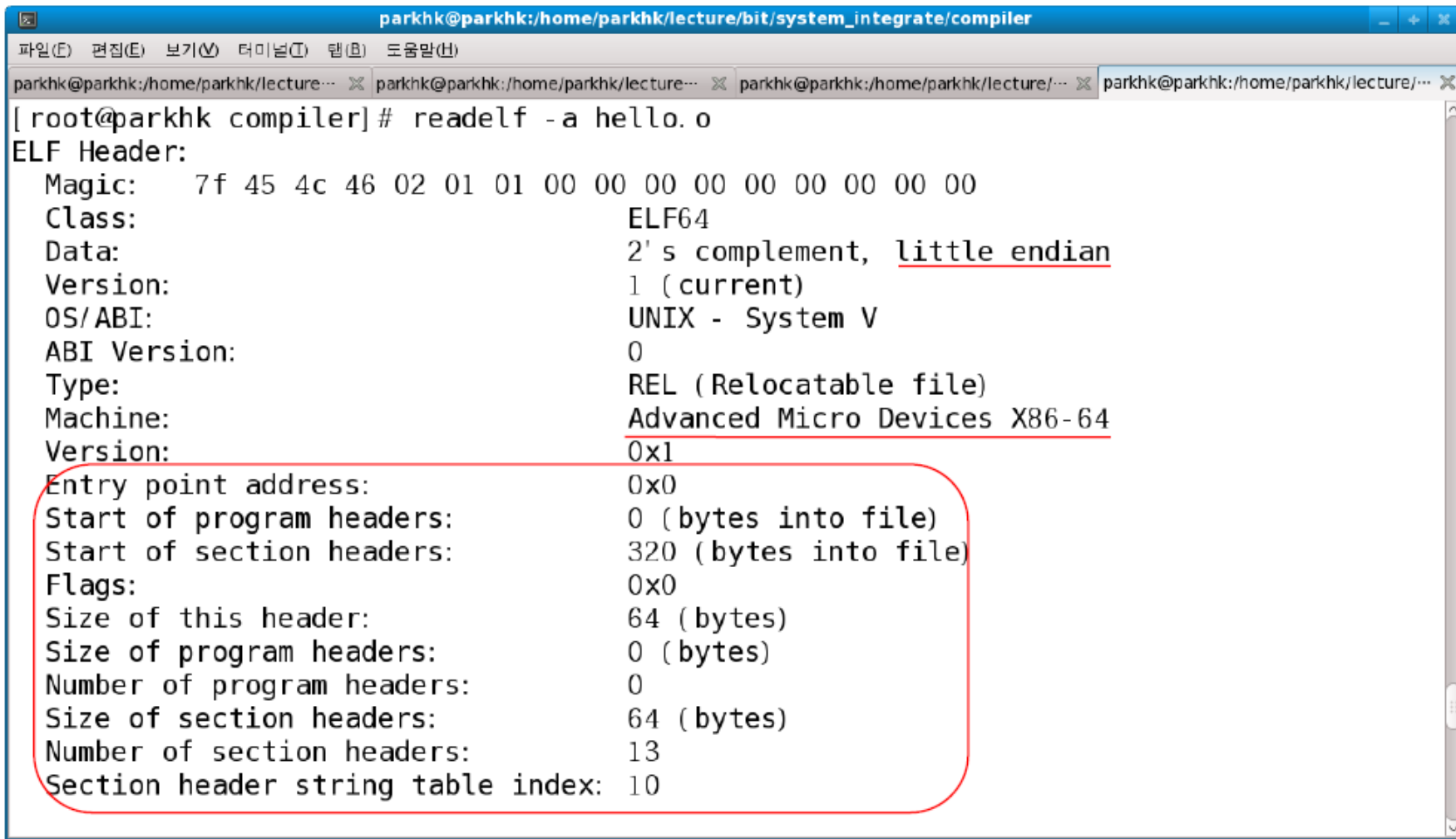
- `as`에 의해 생성되는 `.o`파일은 어셈블된 `.s`의 인스트럭션과 데이터가 들어 있는 ELF(Executable and Linking Format) 바이너리 포맷 구조를 가짐



source.o 파일 내부구조

GCC 컴파일 과정

■ ELF 헤더 정보

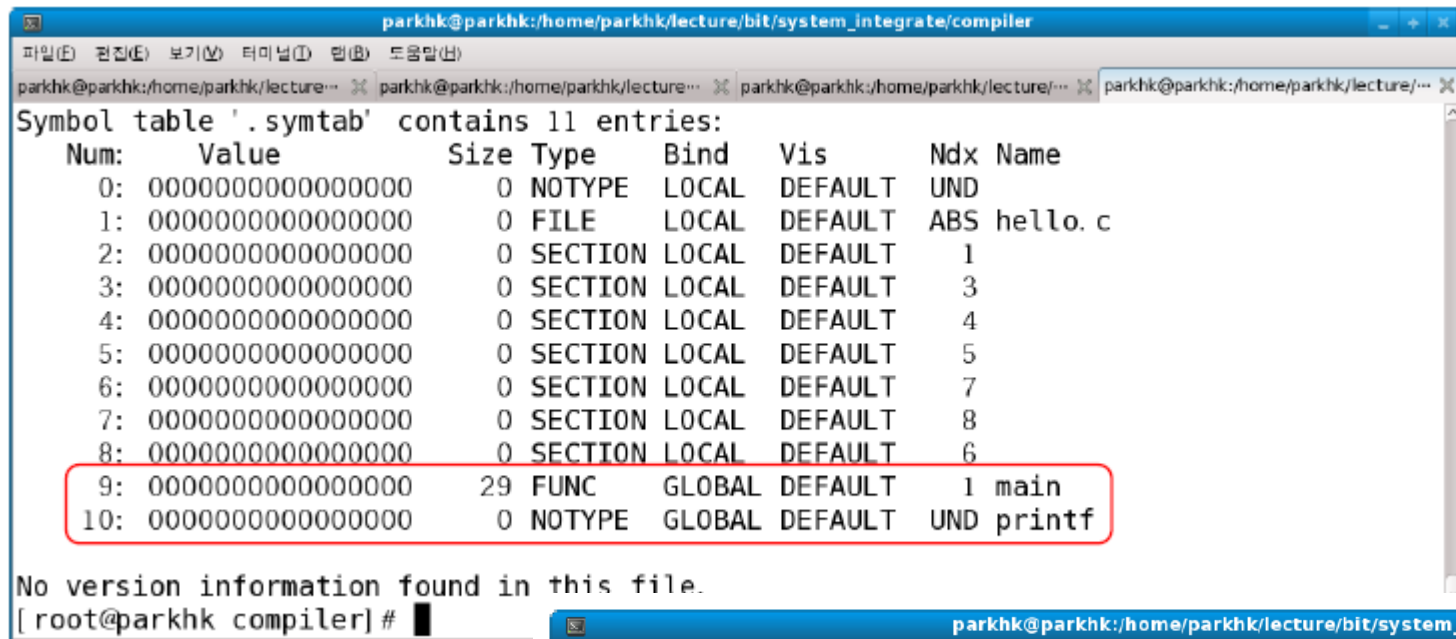


```
parkhk@parkhk:/home/parkhk/lecture/bit/system_integrate/compiler
[root@parkhk compiler]# readelf -a hello.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  REL (Relocatable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x0
  Start of program headers:               0 (bytes into file)
  Start of section headers:               320 (bytes into file)
  Flags:                                  0x0
  Size of this header:                    64 (bytes)
  Size of program headers:                 0 (bytes)
  Number of program headers:                0
  Size of section headers:                 64 (bytes)
  Number of section headers:               13
  Section header string table index:      10
```

GCC 컴파일 과정

■ 심볼 테이블

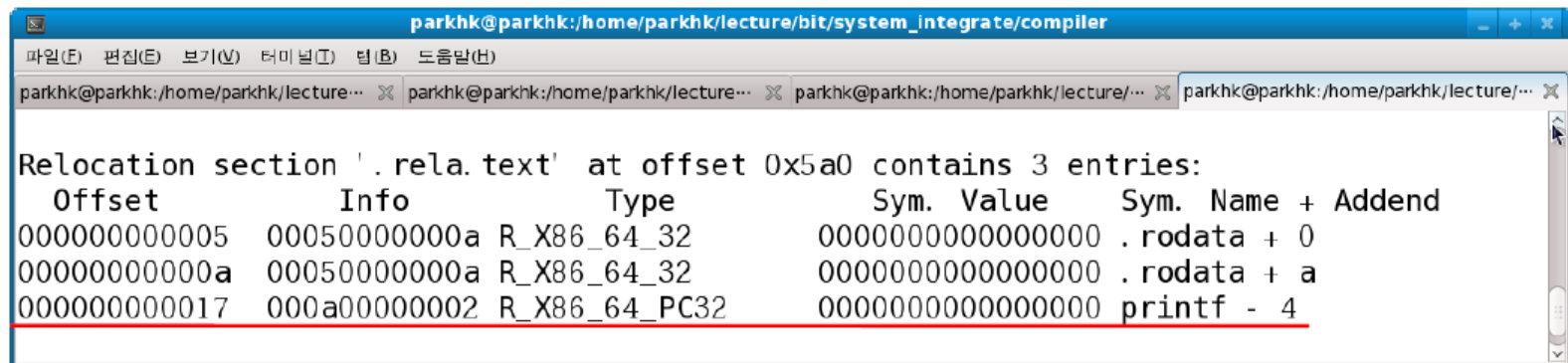
- .c파일 내부에 사용된 심볼들의 정보가 들어 있는 테이블
- 심볼 : 메모리상의 주소를 가지는 모든 것



```
Symbol table '.symtab' contains 11 entries:
Num:      Value              Size Type   Bind   Vis      Ndx Name
  0: 0000000000000000          0 NOTYPE LOCAL DEFAULT UND
  1: 0000000000000000          0 FILE   LOCAL DEFAULT ABS hello.c
  2: 0000000000000000          0 SECTION LOCAL DEFAULT 1
  3: 0000000000000000          0 SECTION LOCAL DEFAULT 3
  4: 0000000000000000          0 SECTION LOCAL DEFAULT 4
  5: 0000000000000000          0 SECTION LOCAL DEFAULT 5
  6: 0000000000000000          0 SECTION LOCAL DEFAULT 7
  7: 0000000000000000          0 SECTION LOCAL DEFAULT 8
  8: 0000000000000000          0 SECTION LOCAL DEFAULT 6
  9: 0000000000000000        29 FUNC    GLOBAL DEFAULT 1 main
 10: 0000000000000000          0 NOTYPE GLOBAL DEFAULT UND printf

No version information found in this file.
[root@parkhk compiler] #
```

■ Relocation섹션 정보

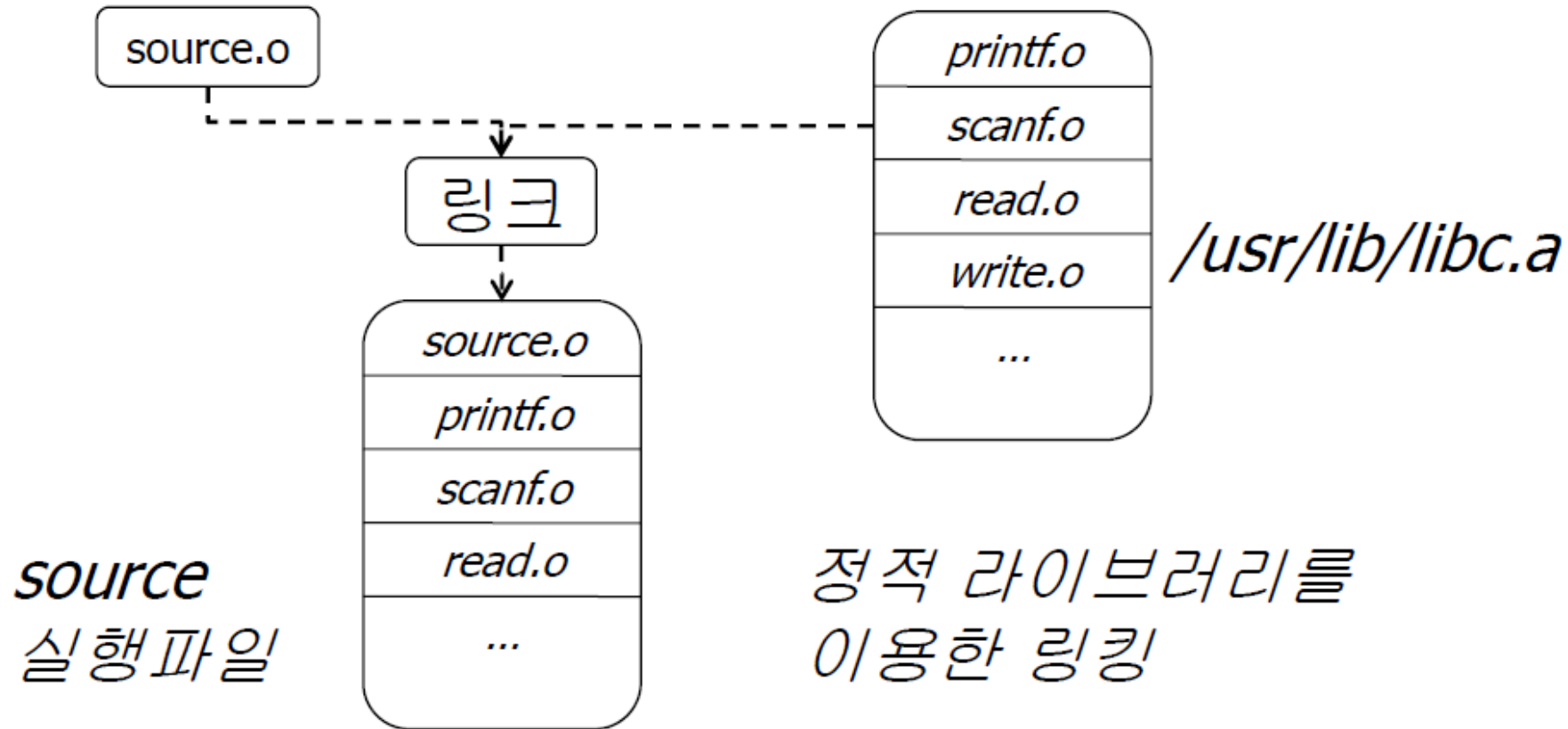


```
Relocation section '.rela.text' at offset 0x5a0 contains 3 entries:
Offset             Info             Type              Sym. Value      Sym. Name + Addend
0000000000000005  000500000000a R_X86_64_32      0000000000000000 .rodata + 0
000000000000000a  000500000000a R_X86_64_32      0000000000000000 .rodata + a
0000000000000017  000a000000002 R_X86_64_PC32    0000000000000000 printf - 4
```

GCC 링킹 과정

- collect2에 의한 링킹 과정

ELF 포맷의 바이너리 파일 생성 후 링커에 의해 표준 C 라이브러리와 링크하게 됨



GCC 옵션

-E

전처리 과정의 결과를 화면에 보이는 옵션

-S

cc1으로 전처리된 파일을 어셈블리 파일로 컴파일까지만 수행

.s파일 생성

-c

as에 의한 어셈블까지만 수행하고 링크는 수행하지 않음

.o파일 생성

-v

gcc가 컴파일을 어떤 식으로 수행하는지를 화면에 출력해서 보임

--save-temps

컴파일 과정에서 생성되는 중간 파일인 전처리 파일, 어셈블리 파일, 오브젝트 파일을 현재 디렉토리에 저장
컴파일 과정에서 발견되는 오류 분석 시 이용

GCC 옵션

-I

전처리 과정에서 헤더 파일을 탐색하는 기본 디렉토리를 추가할 때 사용하는 옵션

-include

헤더 파일을 소스 내에 추가할 때 사용

-D

매크로를 외부에서 define 할 때 사용

-U

-D와 반대로 소스 파일 내에 #undef 옵션을 추가한 것과 동일

GCC 옵션

■ 경고 옵션

- Wall 옵션은 모든 모호한 문법에 대한 경고 메시지 출력
- W는 합법적이지만 모호한 코딩에 대해서 부가적인 정보 제공

■ 최적화 옵션

-O0

최적화를 수행하지 않음

-O1

최소 단위 최적화

-O2

가장 많이 사용하는 최적화 옵션으로 일반 응용이나 커널을 컴파일 할 때 사용됨
거의 대부분의 최적화 수행

GCC 옵션

- 최적화 옵션

-O3

가장 높은 레벨의 최적화로 모든 함수를 인라인 함수와 같이 취급

-call instruction을 사용하지 않고 호출

너무나 많은 소스의 변경에 의해 왜곡이 발생할 위험이 있음

-Os

사이즈 최적화 수행

GCC 옵션

- 디버깅 옵션

-g

gdb 에게 제공하는 정보를 바이너리에 삽입

-O와 같이 레벨 지정 가능 (g0~g3)

-pg

프로파일을 위한 코드 삽입

gprof에 의해 실행 파일의 내용을 분석하여 함수 호출 횟수나 수행 시간 분석을 수행

GNU DEBUGGER

■ GDB

- 다양한 아키텍처와 CPU를 위한 낮은 수준의 종속적인 디버깅 까지도 지원
- 임베디드 리눅스 개발 환경에서는 교차-디버거로 컴파일된 GDB를 이용
- 디버거 자체는 개발 호스트 시스템에서 실행되지만, 디버거는 컴파일 시점에 주어진 아키텍처를 위한 바이너리 실행 파일을 인식하도록 되어 있는 것을 의미.

■ 디버깅

- 프로그램의 논리적인 오류를 찾아내는 과정
- 프로그램 수행 중에 내부적인 동작을 파악
- 프로그램 수행 중 오류 발생으로 중단된 원인 파악

GNU DEBUGGER

- C/C++ 디버깅 도구인 GDB debugger
 - 프로그램을 수행하거나 정지 시킬 수 있음
 - 수행 도중 변수 값을 알아 볼 수 있고 프로그램 수행 과정을 추적
 - 프로그램 수행 속도가 늦어 질 수 있음
- GDB를 사용하려면 반드시 gcc 컴파일시에서 **-g** 옵션을 추가함
 - gcc -g file1.c file2.c file3.o
 - file1.c file2.c는 소스 레벨의 디버깅 가능
 - file3.o는 오브젝트 파일을 만들 때 -g로 컴파일 한 경우에 한해서 소스 레벨의 디버깅 가능
 - 최적화 옵션 **-O**와 동시에 사용 가능
 - 최적화는 실행 프로그램에 변동을 가져옴
 - 예: 변수 또는 코드가 사라 질 수 있음
 - 추천: 최적화 없이 우선 완전하게 동작하는 프로그램 개발 후 최적화 적용
 - 최적화 사용시 프로그램이 비정상적으로 동작할 수 있음을 명심

GDB 사용

■ gdb 명령으로 시작

- gdb ex-prog core-dump
- 실행 파일 'ex-prog' 를 디버깅
- 프로그램 오류에 의하여 중단된 프로세스의 덤프 파일 'core-dump' 를 디버깅

■ gdb 옵션들

- -d dir : 소스 파일이 있는 디렉토리 지정
- -x file : gdb 시작 후 file에 있는 gdb 명령어 우선 실행
- -c number : 실행 중인 프로세스의 pid가 number인 프로세스를 디버깅
- -batch : -x file로 지정된 gdb 명령 수행 후 gdb 종료

■ gdb 종료

- 명령어 모드에서 quit 또는 q

GDB 명령 기본

- gdb 몇 개만 사용해도 디버깅을 잘 할 수 있음
- 프로그램을 수행하고 정지 시키고 그리고 변수 값을 알아 보는 일이 대부분임
- 명령어를 잘 모른 다면 help 명령어 사용
- 명령어 입력 중 TAB 키를 사용하면 가능한 입력 나열
만약 가능한 입력이 하나 밖에 없으면 자동으로 명령어를 확장해 줌

GDB 명령어: 파일 내용 출력

- **list** 명령어는 실행하고 있는 부분의 소스를 출력

(gdb) list

- 프로그램 실행 전에는 main 함수의 내용을 보여 줌

- 특정 부분을 보려면 소스 프로그램의 줄 번호를 입력함

(gdb) list 90

- 또는 특정 함수의 소스 코드를 보려면 함수의 이름 입력

(gdb) list badfunc

GDB 명령어: 프로그램 수행

- **run** 명령어: 디버그 할 프로그램 수행

프로그램에 전달 할 인자 지정 가능

(gdb) run -b < invalues > outtable

- 다음 run 명령어에서 인자를 주지 않으면 이전의 인자 그대로 사용

- **set arg** 명령을 사용하여 인자를 바꾸거나 설정할 수 있음

(gdb) set args -b < invalues > outtable

- show args로 설정된 인자를 알아 볼 수 있음

(gdb) show args

- **실행 디렉토리 변경**

GDB의 실행 디렉토리를 이어 받으나 cd 명령으로 바꿀 수 있음

GDB 명령어: 프로그램 수행

- 환경 변수 설정 변경
GDB의 환경 변수를 이어 받으나 `set environment` 명령으로 바꿀 수 있음
- 터미널 입출력
GDB의 터미널 입출력을 이어 받으나 `tty` 명령으로 바꿀 수 있음
- 이미 실행 중인 프로세스
`Attach`, `detach` 명령으로 이미 실행 중인 프로세스 디버깅 가능
- 다중 스레드 디버깅
`Info thread` 명령어로 스레드를 알아보고 `thread number` 명령어로 특정 스레드 디버깅 가능

GDB 명령어: 프로그램 정지

- **break** 명령어로 프로그램 정지 지점 설정

(gdb) break *line-number*

소스의 특정 위치인 주어진 줄, line-number에 오면 프로그램 정지

(gdb) break *func-name*

소스의 특정 함수, func-name에 오면 프로그램 중지

(gdb) break *line-or-func* if *condition*

소스의 특정 위치에서 조건, condition을 만족하면 프로그램 정지

GDB 명령어: 프로그램 정지

- **watch** 명령어로 특정 조건을 만족하면 프로그램 정지

(gdb) *watch condition*

- 프로그램의 속도가 상당히 느려짐
- 일반적으로 **watch**를 사용하여 프로그램 오류를 대략적인 위치를 알아내고 **break**를 사용하여 정확한 위치를 추적함

GDB 명령어: 프로그램 상태 파악

- 프로그램을 정지시켜 놓고 상태를 파악하여 오류를 알아냄
 - whatis 명령어로 변수의 타입을 알아 봄
(gdb) whatis variable
 - print 명령어로 변수의 값을 알아 봄
(gdb) print variable
- 고급 print 명령어 (con' t)
 - print 명령어의 variable은 C 언어의 구문을 그대로 사용할 수 있음
(gdb) print a->member

GDB 명령어: 프로그램 상태 파악

■ 고급 print 명령어

함수의 결과 값도 함수를 호출하여 알아 볼 수 있음

```
(gdb) print function(args,...)
```

** 전역 변수가 변경 될 수 있음

@을 사용하여 배열의 길이를 지정할 수 있음

```
(gdb) print *p@10
```

::를 사용하여 범위(scope)를 지정할 수 있음

```
(gdb) print func::var-x
```

/x,u,o,c,etc 를 사용하여 출력 형태를 지정할 수 있음

```
(gdb) print/x var-x
```


GDB 명령어: 정지점 관리

- 복수개의 정지점(breakpoint) 설정 가능
- 정지점은 **info breakpoints** 를 사용하여 설정된 정지점 알아 볼 수 있음
- **delete break-number** 명령을 사용하여 정지점 제거 가능
인자를 주지 않으면 모든 정지점 지움
- **disable, enable**을 사용하여 정지점을 잠시 유효 및 무효화 시킬 수 있음

GDB 명령어: 계속 실행

- **continue** 명령어를 사용하여 프로그램 계속 수행 가능
- **step** 명령어를 통해서 한 줄씩 수행 가능
함수를 만나면 함수로 들어 가서 한 줄 수행
- **next** 명령어를 통해서 한 줄씩 수행 가능
함수도 한 줄로 인식하여 함수 안으로 들어가지 않음
- **finish** 현재 실행 중인 함수의 끝까지 실행하고 멈춤
- **return value** 은 현재 실행 중인 함수를 취소하고 **value**를 리턴 값으로 사용함

GDB 명령어: 명령어 자동 실행

- command 명령어를 사용하여 정지시 자동으로 실행하는 명령어 설정
(gdb) command break-number
...list-of-command
...list-of-command
End
- display 명령어를 사용하여 프로그램이 멈출 때마다 변수 값 출력
(gdb) display variable

GDB 명령어: 시그널 설정

- handle 명령어를 사용하여 디버깅 프로그램에 보내는 시그널 제어
(gdb) handle signal action-list
 - signal은 SIGINT와 같은 시스템에서 정한 시그널
 - Action-list는 다음과 같은 의미를 가짐
 - nostop: 시그널을 받았을 때 프로그램에 넘겨주지 않고 정지 시키지도 않음
 - stop: 시그널을 받았을 때 프로그램을 정지 시키고 디버그 모드로 전환
 - print: 시그널을 받았을 때 메시지 출력
 - pass: 프로그램에 시그널을 넘겨 줌
 - nopass: 프로그램에 시그널을 넘겨주지 않음
 - 예 : (gdb) handle SIGINT stop print
- singal 명령을 통하여 프로그램에 시그널 발생
(gdb) signal 2

GDB 명령어 : 정리

list	현재 위치에서 소스 파일의 내용을 10줄 보여준다 list 2, 15 : 소스 파일의 2 ~ 15 까지를 보여준다.
run	프로그램을 시작한다.(break가 있다면 break까지 실행) run arg : 새로운 인수를 가지고 프로그램을 시작한다. arg는 “*”나 “[...]”를 포함할 수도 있다. 셸의 사용까지도 확장될 수 있다. “<”, “>”, “>>”같은 입출력 방향 재지정기호도 또한 허용된다.
break	특정 라인이나 함수에 정지점을 설정한다. break function : 현재 파일 안의 함수 function에 정지점을 설정한다. break file:function : 파일file안의 function에 정지점을 설정한다. watch : 감시점 설정(감시점은 어떤사건이 일어날 때에만 작동한다) until : 실행중 line까지 실행
clear	특정 라인이나 함수에 있던 정지점을 삭제한다.
delete	몇몇 정지점이나 자동으로 출력되는 표현을 삭제한다.
next	다음 행을 수행한다. 서브루틴을 호출하면서 계속 수행한다. 호출이 발생하지 않으면 step와 같다. next n : 이를 n번 수행하라는 의미

GDB 명령어 : 정리

step	한 줄씩 실행 시킨다. 함수를 포함하고 있으면 함수 내부로 들어가서 한 줄씩 실행시킨다.
print	print expr : 수식의 값을 보여준다.
display	현재 display된 명령의 목록을 보여준다.
bt	프로그램 스택을 보여준다. (backtrace)
kill	디버깅 중인 프로그램의 실행을 취소한다.
file	file program : 디버깅할 프로그램으로서 파일을 사용한다.
cont	continue : 현재 위치에서 프로그램을 계속 실행한다.
help	명령에 관한 정보를 보여주거나 일반적인 정보를 보여준다.
quit	gdb에서 빠져나간다.

CORE 파일 분석

- 예기치 못한 일이 발생하여 비정상적인 종료가 발생할 때 운영체제는 디스크에 **core file**을 남김
- 프로세스의 메모리 이미지를 **dump**한 것
- **core file**을 **gdb**와 함께 사용하여 프로그램의 상태를 조사하고 실패 원인을 규명할 수 있음
-
- 메모리에 관한 문제는 **checker** 패키지를 사용하여 예방 가능함
- 메모리 **fault**를 일으키는 경우에는 메모리가 충돌하면서 파일을 **dump**함
- **core file** 은 일반적으로 프로세스를 실행시킨 현재 작업 디렉터리에 생성되지만 프로그램 내에서 작업 디렉터리를 바꾸는 경우도 있음

CORE 파일 분석

- 일반적으로 Linux는 부팅 시에 core file 을 만들지 않도록 설정되어 있음
- core file 생성을 가능케 하려고 한다면 그것을 다시 가능케 하는 셸의 내장 명령을 사용함
\$ ulimit -c unlimited
- gdb 에서 코어 파일을 이용한 디버깅
gdb 실행파일명 core파일명

실행 중인 프로그램 디버깅하기

- gdb는 이미 실행 중인 프로그램도 디버깅할 수 있음
- 프로세스 실행을 가로채고 조사한 뒤 다시 원래 상태로 실행하도록 할 수 있음
- attach 명령을 사용하여 실행 중인 프로세서에 gdb를 붙임
- attach 명령을 사용하기 위해서는 프로세스에 해당하는 실행 프로그램에 허가권을 가지고 있어야 함
- 예를 들어 프로세스 ID 254번으로 실행 중인 pgmseq 프로그램이 있다면 다음과 같이 입력
gdb pgmseq
attach 254
gdb pgmseq 254

실행 중인 프로그램 디버깅하기

- 일단 **gdb**가 실행 중인 프로세스에 부착되면 프로그램을 일시 중지 시키고 **gdb**명령을 사용할 수 있도록 제어권을 가져온 옴
- **break**를 사용하여 중지점을 사용할 수 있고 중지점에 이를 때까지 실행하도록 **continue** 명령을 사용할 수 있음
- **detach**명령을 사용하여 **gdb**를 실행 중인 프로세스에서 떼어 냄
- 필요에 따라 다른 프로세스에 대하여 **attach**명령을 사용할 수 있음

GDB DEBUGGER 실습

% gdb

gdb를 먼저 실행 후 file이라는 명령으로 program을 부름

% gdb program

일반적인 방법

% gdb program core

코어파일을 사용할 때 동시에 인자로 줌

% gdb program 1234

실행중인 프로세스를 디버그 하려면 프로세스 ID를 두 번째 인자로 주면 된다.

이 명령은 gdb를 ('1234' 란 이름의 파일이 없다면) 프로세스 1234에 접속시킨다.

(gdb는 core파일을 먼저 찾는다.)

GDB DEBUGGER 실습

■ 예제 소스 1

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int i;
6     double j;
7
8     for(i=0;i<5;i++) {
9         j=i/2+i;
10        printf("j is %f \n", j);
11    }
12    return 0;
13 }
```

\$ gcc -g test.c -o test

\$ gdb test

GDB DEBUGGER 실습

(gdb) list // list는 소스 내용을 **10**줄씩 보여준다.

```
(gdb) list
1      #include <stdio.h>
2
3      int main()
4      {
5          int i;
6          double j;
7
8          for(i=0;i<5;i++) {
9              j=i/2+i;
10             printf("j is %f \n", j);
(gdb)
```

(gdb) b 8 // break 8 : line 8에 breakpoint를 잡는다.

```
(gdb) b 8
Breakpoint 1 at 0x80483d5: file test.c, line 8.
```

GDB DEBUGGER 실습

(gdb) r // breakpoint 전까지 프로그램 실행

```
(gdb) r
Starting program: /root/program/test

Breakpoint 1, main () at test.c:8
8           for(i=0;i<5;i++) {
Missing separate debuginfos, use: debuginfo-install glibc.i686
```

(gdb) s // step : 한 줄 실행시킨다.

```
(gdb) s
9           j=i/2+i;
```

(gdb) s

```
(gdb) s
10          printf("j is %f \n", j);
```

(gdb) p j // print j : j의 값을 본다.

```
(gdb) p j
$1 = 0
```

GDB DEBUGGER 실습

(gdb) n // next

(gdb) display i // 변수 i를 display 목록에 추가

(gdb) display j // 변수 j를 display 목록에 추가

(gdb) n // next

(gdb) quit // 종료

```
(gdb) n
j is 0.000000
8           for(i=0;i<5;i++) {
(gdb) display i
1: i = 0
(gdb) display j
2: j = 0
(gdb) n
9           j=i/2+i;
2: j = 0
1: i = 1
(gdb) quit
The program is running.  Exit anyway? (y or n) y
```

GDB DEBUGGER 실습

■ 예제 소스 2

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a, b, dab;
6     printf("input num a, b : ");
7     scanf("%d %d", &a, &b);
8     dab = hab(a,b);
9     printf("\n %d + %d = %d \n", a, b, dab);
10 }
11
12 int hab(int x, int y)
13 {
14     return (x+y);
15 }
```

- 이 예제는 여러 곳에서 호출되는 함수 안에서 충돌이 일어날 경우 함수가 어디로부터 호출되었는지 그리고 어떤 상황에서 충돌이 일어났는지 파악하고자 함

- **backtrace (bt)** 명령을 이용하면 충돌이 일어난 시점에서 프로그램의 현재 호출 스택(call stack) 상태를 볼 수 있음.

- 호출 스택은 현재 함수까지 이르는 호출 목록이다. 함수를 호출할 때마다 보관된 레지스터 값, 함수 전달 인수, 지역 변수 등의 자료를 스택에 push한다. 이렇게 해서 각 함수들은 스택상에 일정 공간을 차지한다.

- 특정함수에 대하여 스택에서 사용되고 있는 메모리 부분을 스택프레임(frame)이라 부르며 호출 스택은 이러한 스택 프레임을 순서대로 정렬한 목록이다

GDB DEBUGGER 실습

(gdb) b 8

(gdb) r // 숫자 3 4 입력

```
(gdb) b 8
Breakpoint 1 at 0x804842b: file hab.c, line 8.
(gdb) r
Starting program: /root/program/hab
input num a, b : 3 4

Breakpoint 1, main () at hab.c:8
8      dab = hab(a,b);
Missing separate debuginfos, use: debuginfo-install glibc.i686
```

(gdb) bt // 현재 스택에 main이 있다.

```
(gdb) bt
#0  main () at hab.c:8
```

(gdb) s

```
(gdb) s
hab (x=3, y=4) at hab.c:14
14      return (x+y);
```

(gdb) bt // 지금은 스택에 hab이 있다.

```
(gdb) bt
#0  hab (x=3, y=4) at hab.c:14
#1  0x0804843d in main () at hab.c:8
```

(gdb) frame 0 // hab의 상태를 점검하기 위해서 스택 프레임 0번으로 이동

```
(gdb) frame 0
#0  hab (x=3, y=4) at hab.c:14
14      return (x+y);
```

GDB DEBUGGER 실습

(gdb) up // hab이 어떻게 호출되었는가를 보기 위하여 상위 스택프레임으로 이동

```
(gdb) up
#1 0x0804843d in main () at hab.c:8
      dab = hab(a,b);
```

(gdb) info program // 프로그램의 실행 상태를 보여 준다.

```
(gdb) info program
Using the running image of child process 3192.
Program stopped at 0x804846d.
It stopped after being stepped.
```

(gdb) info locals // 현재 함수 내에서 모든 지역 변수 이름과 값을 출력한다.

```
(gdb) info locals
a = 3
b = 4
dab = 2230592
```

(gdb) info variables // 소스파일 순서대로 프로그램 내에 알려져 있는 모든 변수를 출력한다.

```
(gdb) info variables
All defined variables:

Non-debugging symbols:
0x08048544 __fp_hw
0x08048548 __IO_stdin_used
0x0804854c __dso_handle
0x080485ec __FRAME_END__
0x080495f0 __CTOR_LIST__
0x080495f0 __init_array_end
0x080495f0 __init_array_start
0x080495f4 __CTOR_END__
0x080495f8 __DTOR_LIST__
0x080495fc __DTOR_END__
0x08049600 __JCR_END__
0x08049600 __JCR_LIST__
0x08049604 __DYNAMIC__
0x080496d0 __GLOBAL_OFFSET_TABLE__
0x080496ec __data_start
0x080496ec data_start
0x080496f0 dtor_idx.5647
---Type <return> to continue, or q <return> to quit---
```

GDB DEBUGGER 실습

예제 소스 3

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char *bug=NULL;
6
7     strcpy(bug, "debug");
8     printf("bug is %s \n", bug);
9
10    return 0;
11 }
```

\$ gcc -g coredebug.c -o coredebug

\$./coredebug

→ 세그먼트 오류가 발생된 후 core 파일이 생성됨

GDB DEBUGGER 실습

(gdb) b 7

(gdb) r

(gdb) p bug

```
(gdb) b 7
Breakpoint 1 at 0x80483dc: file coredebug.c, line 7.
(gdb) r
Starting program: /root/program/coredebug

Breakpoint 1, main () at coredebug.c:7
7          strcpy(bug,"debug");
Missing separate debuginfos, use: debuginfo-install glibc.i686
(gdb) p bug
$1 = 0x0
```

gdb 에서 **0x0**는 **NULL**이다. 즉 번지가 없다. (gdb) s

```
(gdb) s

Program received signal SIGSEGV, Segmentation fault.
0x080483df in main () at coredebug.c:7
7          strcpy(bug,"debug");
```

line 7번 strcpy에서 **segmentation fault**가 발생한 것을 알 수 있다.

bug에 번지를 할당하면 에러를 해결할 수 있다.

GDB DEBUGGER 실습

\$ gdb ./coredebug core

```
[root@esp program]# gdb coredebug core.4253
GNU gdb Red Hat Linux (6.6-45.fc8rh)
Copyright (C) 2006 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "i386-redhat-linux-gnu"...
Using host libthread_db library "/lib/libthread_db.so.1".

warning: core file may not match specified executable file.

warning: Can't read pathname for load map: Input/output error.
Reading symbols from /lib/libc.so.6...done.
Loaded symbols for /lib/libc.so.6
Reading symbols from /lib/ld-linux.so.2...done.
Loaded symbols for /lib/ld-linux.so.2
Core was generated by `./core'.
Program terminated with signal 11, Segmentation fault.
#0  0x080483df in main () at coredebug.c:7
7      strcpy(bug, "debug") ;
```

gdb는 signal 11 번과 함께 core file일이 생성되었음을 알려 준다. 여기서는 허가 받지 않은 메모리 공간에 읽기, 쓰기를 시도했기 때문에 커널이 프로세스에게 signal 11을 보냈다. 이 시그널로 인해 프로세스는 종료하면서 코어 파일을 덤프한다.

GDB DEBUGGER 실습

■ 기타 기능

프롬프트 변경

- (gdb) set prompt haha: //haha:로 프롬프트 변경

리스트 라인 수 변경

- (gdb) set listsize 20 // list 출력 라인 수를 20으로 변경

특정한 부분으로 이동

- (gdb) | main // 문자 패턴 main으로 이동
- (gdb) | 25 // 25라인으로 이동
- (gdb) | file.c:func // 파일명 file.c의 func이란 함수로 이동

브레이크 포인트 추가

- (gdb) b 19 // line 19에 breakpoint 추가
- (gdb) b main // 문자패턴 main에 breakpoint 추가
- (gdb) b file.c:10 // file.c의 10번째 줄에 breakpoint 추가
- (gdb) b -2 // 현재 기준 -2번째 줄에 breakpoint 추가
- (gdb) b 10 if tmp ==0 // tmp 값이 0일 때 10번째 줄에 breakpoint 추가..

브레이크 포인트 삭제

- (gdb) cl hello
- (gdb) cl 10
- (gdb) cl file.c:20
- (gdb) d // 모든 breakpoint 삭제

GDB DEBUGGER 실습

■ 기타 기능

브레이크 포인트 확인

- (gdb) info breakpoints // 현재 breakpoint 확인

브레이크 포인트 활성화/비활성화

- (gdb) enable 2
- (gdb) disable 2

프로그램 수행 명령

- (gdb) r (run) // 프로그램 수행
- (gdb) r arg1 arg2 // 프로그램 수행 시 arg1과 arg2를 전달
- (gdb) k (kill) // 프로그램 수행 종료
- (gdb) s (step) // 현재 행 수행 후 정지, 함수 호출 시 함수 내부로 들어감
- (gdb) n (next) // 현재 행 수행 후 정지, 함수 호출 시 함수 수행 후 다음 행으로 감

(gdb) c (continue) // 브레이크 포인트를 만날 때까지 계속 진행

(gdb) u (until) // 현재 루프를 빠져나감

(gdb) finish // 현재 함수를 수행하고 빠져나감

(gdb) return // 현재 함수를 수행하지 않고 빠져나감

(gdb) return arg1 // 현재 함수를 수행하지 않고 빠져나가는데... arg1을 리턴

(gdb) si // 현재의 instruction을 수행, 함수 호출 시 함수 내부로 들어감

(gdb) ni // 현재의 instruction을 수행, 함수 호출 시 함수 내부로 들어가지 않음

GDB DEBUGGER 실습

■ 기타 기능

와치 포인트

- 변수 값 이 바뀔 때마다 브레이크가 걸리면서 변수의 이전 값(old value)과 현재값(new value)을 출력한다.
- (gdb) watch i // 변수 i에 와치 포인트를 설정.

변수와 레지스터 값 출력

- (gdb) info locals // 현재 상태에서의 지역변수 출력
- (gdb) info variables // 현재 상태에서의 전역변수
리스트 출력
- (gdb) p (printf) // 개별 변수 값 출력
- (gdb) p *pointer // pointer 변수 값 출력
- (gdb) p *pointer@4 // pointer 구조체 배열 값 출력
@ 다음에 오는 숫자는 배열의 크기
- (gdb) p \$eax // 레지스터 eax 값 출력

GDB DEBUGGER 실습

■ 기타 기능

display를 이용한 변수 값 출력

- 특정 변수 값을 **display** 목록에 추가하게 되면 **p** 명령어로 매번 확인하는 불편함 없이 **s** 명령어로 자동으로 확인
- 변수의 범위를 벗어나면 자동으로 출력 중지
- (gdb) display i // 변수 i의 값을 **display** 목록에 추가
- (gdb) undisplay i // 변수 i의 값을 **display** 목록에서 삭제
- (gdb) enable display 3 // 디스플레이 번호 3번을 활성화
- (gdb) disable display 4 // 디스플레이 번호 4번을 비활성화

인스트럭션 레벨 디버깅

- (gdb) disass play //play함수에 대한 디스어셈블리.
- (gdb) display \$pc //현재의 instruction을 보여줌
- \$pc는 gdb내부 변수로서 현재 instruction의 위치를 가리키는 Program Counter(PC) 레지스터

원격 디버깅

gdb/gdbserver를 이용한 원격 디버깅

